

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

ERROR ANALYSIS TASK

Name of the Student	P. Hithaishi
Reg. No	192425196

COURSE NAME: Deep Learning
FACULTY: Dr. Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

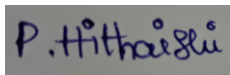
Duration: 30 minutes

Marks: 25

Code of Conduct:

I P. Hithaishi (192425196) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

QUESTIONS:

Total Marks:25

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

a) Identify the conceptual error in the likelihood formulation.

Answer :

The conceptual error is that the likelihood function is written as the **sum of probabilities** instead of the **product of probabilities**.

For independent observations, the likelihood should represent the joint probability of observing all data points. Since the observations are independent, their probabilities must be **multiplied**, not added.

Incorrect formulation:

$$L(\theta) = \sum_{i=1}^n P(x_i | \theta)$$

Correct formulation:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

Using the sum does not represent the joint likelihood and leads to an incorrect optimization objective.

b) Explain how this mistake affects parameter estimation.

Answer:

Writing the likelihood as a sum causes the optimization process to maximize the wrong objective function.

Effects include:

- Incorrect parameter estimates.
- Failure to identify the true model parameters.
- Reduced prediction accuracy.
- The estimator becomes inconsistent even with a large dataset.

- The classifier may misclassify new samples because the learned parameters do not maximize the true probability of the observed data.

Thus, increasing the amount of data cannot compensate for an incorrect likelihood formulation.

c) Suggest the correct mathematical formulation and reasoning.

Answer :

For independent observations $x_1, x_2, \dots, x_n$, the correct likelihood function is

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

Taking the logarithm gives the log-likelihood:

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

Reasoning:

- Multiplication represents the joint probability of independent observations.
- Maximizing the likelihood finds the parameter values that make the observed data most probable.
- The log transformation converts multiplication into addition without changing the location of the maximum.

d) How would using log-likelihood fix computational issues?

Answer:

The log-likelihood simplifies optimization and improves numerical stability.

Advantages:

- Converts products into sums, making calculations easier.
- Prevents numerical underflow caused by multiplying many small probabilities.
- Simplifies differentiation during gradient-based optimization.
- Reduces computational complexity.
- Produces the same optimal parameter estimates as the original likelihood because the logarithm is a monotonic increasing function.

Therefore, log-likelihood is preferred in practical machine learning algorithms such as Logistic Regression, Naïve Bayes, and Maximum Likelihood Estimation.

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

a) List possible implementation errors in the learning algorithm.

Answer :

If a Perceptron fails to converge on a **linearly separable dataset**, the implementation may contain one or more of the following errors:

- Incorrect weight update formula.
- Bias term is omitted or updated incorrectly.
- Wrong activation function or decision threshold.
- Incorrect calculation of the prediction error.
- Learning rate is set improperly.
- Training data is not shuffled between epochs.
- Incorrect initialization of weights.
- Errors in data preprocessing or feature scaling.
- Incorrect stopping criteria or insufficient training iterations.

These implementation mistakes prevent the Perceptron from learning the correct decision boundary.

b) Analyze how incorrect learning rate or weight updates affect convergence.

Answer:

- A very high learning rate causes large weight changes, making the model overshoot the correct decision boundary and preventing convergence.
- A very low learning rate results in very small updates, making training extremely slow.
- Incorrect weight update equations move the decision boundary in the wrong direction, increasing classification errors instead of reducing them.
- An incorrectly updated bias term shifts the decision boundary improperly, reducing model accuracy.
- As a result, the perceptron may fail to reach a stable solution even for linearly separable data.

c) Identify dataset-related issues that may cause this problem.

Answer:

Possible dataset-related issues include:

- Incorrect or inconsistent class labels.
- Duplicate or conflicting samples with different labels.
- Presence of noisy or mislabeled training examples.
- Features with very different scales causing unstable updates.
- Missing or invalid feature values.
- Extremely imbalanced class distribution.
- Data preprocessing errors such as incorrect normalization or encoding.

d) Propose modifications to ensure convergence.

Answer:

To ensure convergence:

- Verify the correct implementation of the perceptron weight update rule:

$$w = w + \eta(y - \hat{y})x$$

- Include and correctly update the bias term.
- Encode class labels consistently as +1 and -1.
- Choose an appropriate learning rate (e.g., 0.01–0.1).
- Normalize or standardize input features before training.
- Shuffle the training data in every epoch.
- Remove mislabelled or inconsistent samples from the dataset.
- Monitor training error and stop when no misclassifications occur or when the maximum number of iterations is reached.

Conclusion:

For a linearly separable dataset, a correctly implemented perceptron with proper preprocessing, appropriate learning rate, and accurate weight updates is guaranteed to converge according to the Perceptron Convergence Theorem.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

a) Identify possible mathematical or coding errors in gradient computation.

Answer :

A Multilayer Perceptron (MLP) relies on correct gradient computation during backpropagation. If the gradients are computed incorrectly, the loss may increase instead of decrease.

Possible mathematical or coding errors include:

- Incorrect implementation of the chain rule during backpropagation.

- Wrong derivative of activation functions such as Sigmoid, Tanh, or ReLU.
- Incorrect weight update formula (adding gradients instead of subtracting them).
- Bias values are not updated or updated incorrectly.
- Incorrect calculation of the loss function (Mean Squared Error or Cross-Entropy).
- Matrix dimension mismatch during forward or backward propagation.
- Gradients are not reset before each iteration, causing accumulation.
- Learning rate is applied incorrectly in the update equation.
- Incorrect initialization of weights or biases.
- Programming errors such as indexing mistakes, missing loops, or incorrect tensor operations.

Result: Even a small error in gradient computation can prevent the network from learning and cause the training loss to increase continuously.

b) Explain the role of activation functions in gradient flow.

Answer:

Activation functions introduce non-linearity, allowing neural networks to model complex relationships between inputs and outputs. During backpropagation, gradients pass through the derivatives of these activation functions.

- Sigmoid: Produces outputs between 0 and 1. Its derivative becomes very small for large positive or negative inputs, leading to the vanishing gradient problem.
- Tanh: Produces outputs between -1 and 1 . It performs better than Sigmoid because it is zero-centered, but deep networks may still experience vanishing gradients.
- ReLU (Rectified Linear Unit): Outputs the input directly if positive; otherwise outputs zero. It allows gradients to flow efficiently for positive values, making training faster.
- Leaky ReLU: Solves the "dying ReLU" problem by allowing a small gradient even when the input is negative.
- SoftMax: Commonly used in the output layer for multi-class classification to convert outputs into probabilities.

Importance of Activation Functions:

- Enable learning of complex patterns.
- Control gradient flow during backpropagation.
- Affect convergence speed and training stability.

c) Analyze how vanishing or exploding gradients affect training.

Answer:

During backpropagation, gradients are propagated from the output layer to the input layer. If these gradients become extremely small or extremely large, training becomes unstable.

Vanishing Gradient

- Gradients become very close to zero.
- Earlier hidden layers receive almost no weight updates.
- Learning becomes extremely slow or completely stops.
- Common in deep neural networks using Sigmoid or Tanh activations.
- Results in poor accuracy and failure to capture complex features.

Exploding Gradient

- Gradients become excessively large.
- Weight values increase rapidly, causing unstable learning.
- Loss values fluctuate or become very large.
- Training may produce NaN (Not a Number) values due to numerical overflow.
- Model fails to converge.

Effects on Training:

- Slow or unstable convergence.
- Poor model performance.
- Increased computational time.
- Difficulty in learning meaningful features.
- Reduced overall prediction accuracy.

d) Suggest techniques to fix the issue (e.g., normalization, initialization).

Answer:

The following techniques help improve training stability and convergence:

- Use He Initialization for ReLU networks and Xavier Initialization for Sigmoid/Tanh networks.
- Normalize input data using feature scaling or standardization.
- Apply Batch Normalization to stabilize activations during training.
- Use ReLU or Leaky ReLU instead of Sigmoid in deep networks.
- Select an appropriate learning rate or use adaptive optimizers such as Adam or RMSProp.
- Apply Gradient Clipping to prevent exploding gradients.
- Verify the correctness of backpropagation equations and weight updates.
- Use proper regularization techniques (Dropout or L2 Regularization) to improve generalization.

- Monitor training and validation loss to detect issues early.

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

a) Explain why batch gradient descent is slow in large datasets.

Answer :

Batch Gradient Descent (BGD) updates the model parameters only after computing the gradient using the entire training dataset. While this provides an accurate estimate of the gradient, it becomes computationally expensive when the dataset is very large.

Reasons why Batch Gradient Descent is slow:

- It processes all training samples before performing a single weight update.
- Each iteration requires high computational power and memory.
- Training time increases significantly as dataset size grows.
- Weight updates occur less frequently, resulting in slower convergence.
- It is inefficient for real-time or online learning applications.
- Large datasets require longer epochs, delaying model improvement.
- Reading the entire dataset repeatedly increases I/O overhead.
- It is less suitable for applications with continuously arriving data.

Advantages:

- Stable and smooth convergence.
- Accurate gradient estimation.
- Consistent reduction in loss function.

Disadvantages:

- Very slow on large datasets.
- High memory consumption.
- Longer training time.

b) Analyze the cause of fluctuations in SGD.

Answer:

Stochastic Gradient Descent (SGD) updates the model parameters after processing each individual training sample instead of the entire dataset. Because each sample provides a different estimate of the gradient, the optimization path becomes noisy.

Causes of fluctuations:

- Gradient is calculated using only one training example at a time.

- Different samples produce different gradient directions.
- Weight updates occur very frequently.
- Noisy updates cause the loss curve to fluctuate instead of decreasing smoothly.
- High learning rates increase oscillations around the optimal solution.
- Random sampling introduces variance in gradient estimation.

Effects of SGD:

- Faster initial learning.
- Can escape local minima due to random updates.
- Loss graph shows zig-zag or oscillating behaviour.
- May require more epochs to stabilize.
- Final convergence is less smooth than Batch Gradient Descent.

Result:

Although SGD introduces fluctuations, it often reaches a good solution much faster than Batch Gradient Descent.

c) Compare the error behaviour of Mini-Batch Gradient Descent.

Answer:

Mini-Batch Gradient Descent combines the advantages of Batch Gradient Descent and Stochastic Gradient Descent. Instead of using the entire dataset or a single sample, it updates weights using small batches (typically 32, 64, or 128 samples).

Batch Gradient Descent	Stochastic Gradient Descent (SGD)	Mini-Batch Gradient Descent
Uses the entire dataset	Uses one sample	Uses a small batch of samples
Very smooth error curve	Highly fluctuating error curve	Moderately smooth error curve
Slow convergence	Fast convergence	Fast and stable convergence
High computation per update	Low computation per update	Moderate computation
High memory usage	Low memory usage	Moderate memory usage
Accurate gradient	Noisy gradient	Balanced gradient estimate

Error Behaviour:

- Less noisy than SGD.
- Faster than Batch Gradient Descent.
- More stable convergence.

- Efficient GPU utilization.
- Reduced variance in gradient estimates.
- Commonly used in modern deep learning applications.

Result:

Mini-Batch Gradient Descent offers the best balance between convergence speed and stability.

d) Recommend the best approach and justify with error trade-offs.

Answer:

For most machine learning and deep learning problems, Mini-Batch Gradient Descent is the preferred optimization method because it combines the strengths of both Batch Gradient Descent and Stochastic Gradient Descent.

Recommendation: Mini-Batch Gradient Descent

Justification:

- Provides faster convergence than Batch Gradient Descent.
- Produces smoother learning curves than SGD.
- Reduces gradient noise while maintaining computational efficiency.
- Efficiently utilizes CPU and GPU resources.
- Handles large datasets effectively.
- Suitable for deep neural networks and modern optimization algorithms.
- Works well with optimizers such as Adam, RMSProp, and Momentum.

Error Trade-offs

Method	Speed	Stability	Error Fluctuation	Best Use Case
Batch Gradient Descent	Slow	Very High	Very Low	Small datasets
Stochastic Gradient Descent	Fast	Low	High	Online learning
Mini-Batch Gradient Descent	Fast	High	Moderate	Large-scale deep learning

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

a) Explain how high dimensionality increases model error.

Answer:

- High dimensionality means the dataset contains a large number of features.
- As the number of features increases, the feature space grows exponentially.
- Data points become sparse, making it difficult for the model to identify meaningful patterns.
- The model starts learning noise and irrelevant information instead of actual relationships, leading to overfitting.
- More training data is required to represent all possible feature combinations.
- Training time, memory usage, and computational complexity also increase.
- The model performs well on training data but poorly on unseen test data, increasing prediction error.

b) Identify signs of overfitting in this scenario.

Answer:

- Very high training accuracy (e.g., 98–100%) but low testing accuracy.
- Large gap between training and testing performance.
- Training loss decreases continuously, while validation loss starts increasing.
- Poor predictions on new or unseen data.
- The model memorizes training samples instead of learning general patterns.
- Performance changes significantly when evaluated on different datasets.
- High variance and poor generalization indicate overfitting.

c) Analyze why distance-based learning fails in high dimensions.

Answer:

- Distance-based algorithms such as K-Nearest Neighbors (KNN) and K-Means Clustering rely on distance measures to identify similar data points.
- In high-dimensional spaces, distances between all data points become almost equal.
- It becomes difficult to distinguish between the nearest and farthest neighbors.
- Irrelevant and noisy features dominate the distance calculations.

- This reduces the effectiveness of similarity measures.
- As a result, classification and clustering accuracy decrease, leading to poor model performance.

d) Suggest dimensionality reduction or regularization techniques to improve performance.

Answer:

- Apply Principal Component Analysis (PCA) to reduce the number of features while preserving important information.
- Use Linear Discriminant Analysis (LDA) for supervised dimensionality reduction.
- Perform Feature Selection to remove irrelevant or redundant features.
- Apply L1 (Lasso) or L2 (Ridge) Regularization to reduce model complexity.
- Use Dropout and Early Stopping in deep learning models to prevent overfitting.
- Normalize or standardize input features before training.
- Use Cross-Validation for better model evaluation and hyperparameter tuning.
- Increase the size of the training dataset whenever possible.

Conclusion:

The Curse of Dimensionality increases model complexity, causes overfitting, and reduces prediction accuracy. Applying dimensionality reduction techniques such as PCA, LDA, and feature selection, along with regularization methods like L1, L2, Dropout, and Early Stopping, helps improve generalization, reduce model error, and achieve better performance on unseen data.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand